ATLAS

NAVIGATE THE COMPLEXITY OF DATA



Project Atlas

Offline Intelligence and Data Operations Platform

Technical overview · Version 0.2.0 · November 2026

Atlas in one slide

An offline-first platform that turns authorized datasets and documents into **evidence-backed, auditable analysis** — with no cloud dependency at install time or at run time.

- Import, validate, resolve, explore, analyse, report
- Every step keeps a pointer back to the source bytes
- Software proposes; people decide — and are recorded

167
REST API operations

16
UI modules

111
backend tests passing

13
file formats supported

13
data-quality checks

0
cloud services or telemetry

The problem Atlas solves



Messy inputs

Staff lists in CSV, suppliers in Excel, payments from a database, audit reports as PDF, e-mails. The same person is “Dr. Amelia Okafor” in one file and “Amelia Okafor” in another.



Hard constraints

Sensitive data and air-gapped networks rule out cloud analytics. Ad-hoc scripts and spreadsheets lose provenance.



Accountability

Conclusions will be challenged. Every number, link and finding must be traceable, reproducible and independently reviewed.

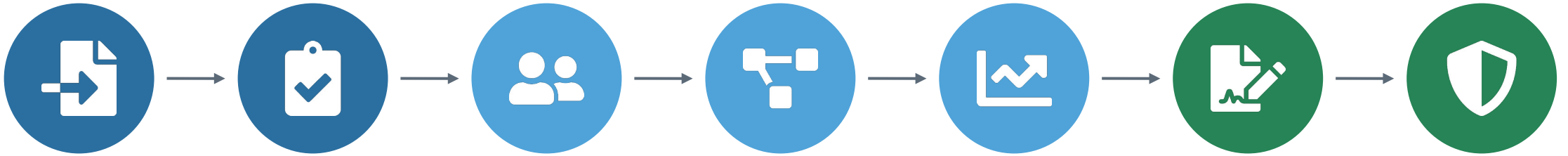
Atlas covers the whole chain — import to auditable report — in one installable product.

SECTION 01

How it works

The end-to-end flow, the architecture and the principles behind it

From raw files to defensible findings



1 **Import**

8 tabular formats, documents, originals hashed

2 **Validate**

Dry run, 13 quality checks, nothing written

3 **Resolve**

Explainable matching, human review, revertible merges

4 **Explore**

Graph, search, timeline, offline maps

5 **Analyse**

Reproducible runs, sandboxed SQL

6 **Investigate**

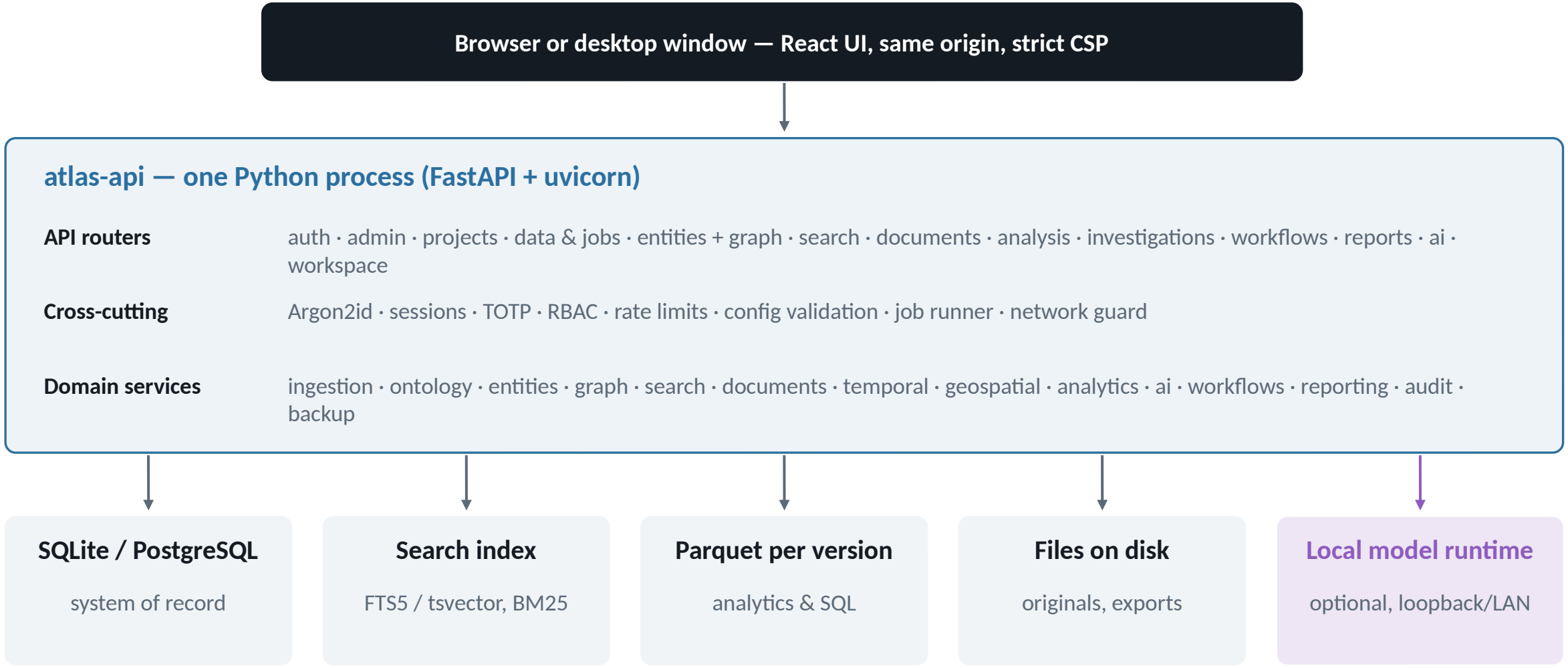
Verbatim evidence, reviewed findings

7 **Report & audit**

Manifests, hash-chained audit log

Provenance at every hop: source file SHA-256 → dataset version → records (raw + normalized) → entities → relationships → analyses → report manifest

One process, five stores



Design principles you can see in the code



Offline-first

Network guard blocks non-local connections; no telemetry; bundled maps and fonts



Evidence first

Raw values kept beside normalized ones; hashes and record links everywhere



Explainable

Versioned, deterministic algorithms with visible inputs and evidence



Human in the loop

Merges, findings, AI suggestions and generated SQL need a person



Reproducible

Every analysis re-runs on recorded inputs and is checked by hash



Tamper-evident

Append-only audit log, hash chain, signed checkpoints

SECTION 02

Capabilities

Ingestion, entity resolution, graph, search, analytics, AI and case work

Every import gets a dry run first

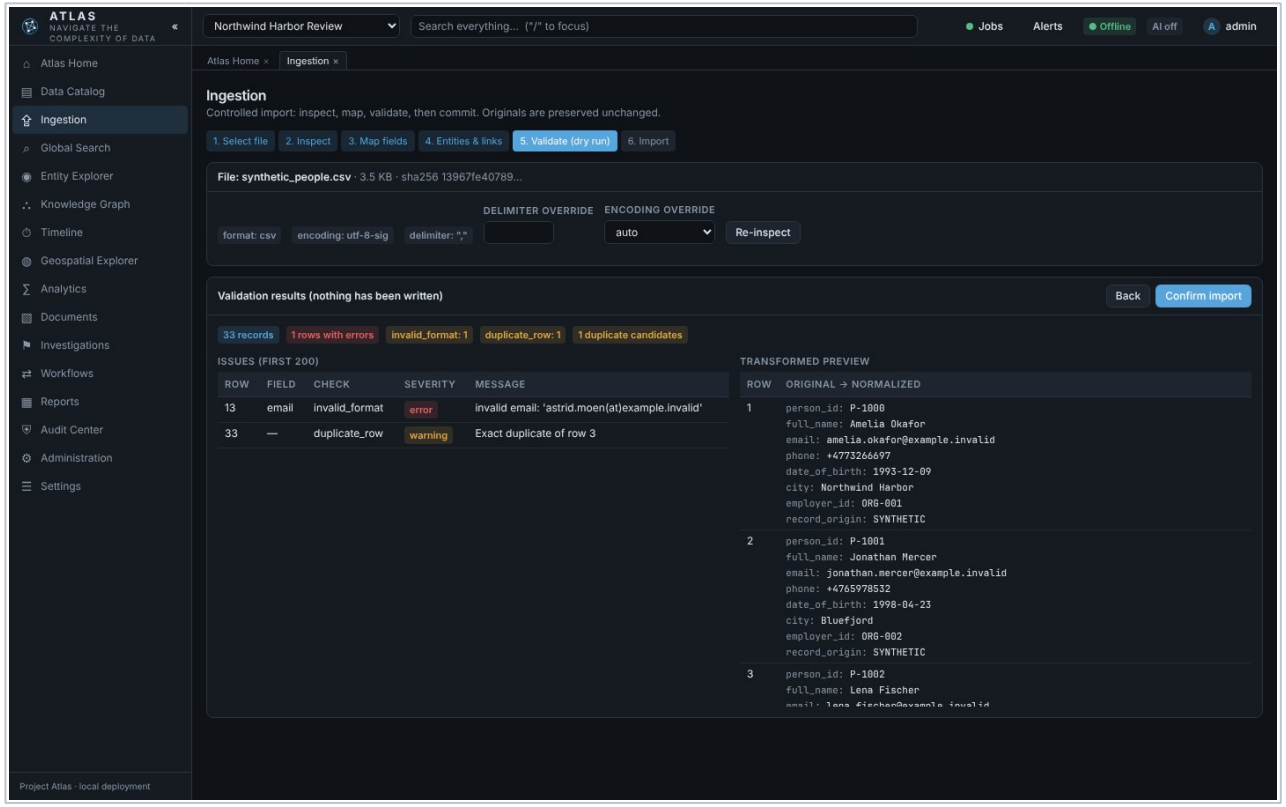
1 — Upload
type allow-list, magic bytes, SHA-256

2 — Inspect
encoding, delimiter, column types

3 — Map
types, time zones, sensitive fields, entities

4 — Dry run
13 quality checks over the whole file

5 — Commit
background job, one transaction



Synthetic transactions: timestamp and coordinate errors found, nothing written.

Originals are kept beside normalized values — reviewers always see what the source said.

Entity resolution that explains itself

Blocking
shared name token or identifier

Compare
only fields both records have

Score
name .45 · identifier .35 · attribute .20

Band
strong ≥ .90 + agreeing identifier

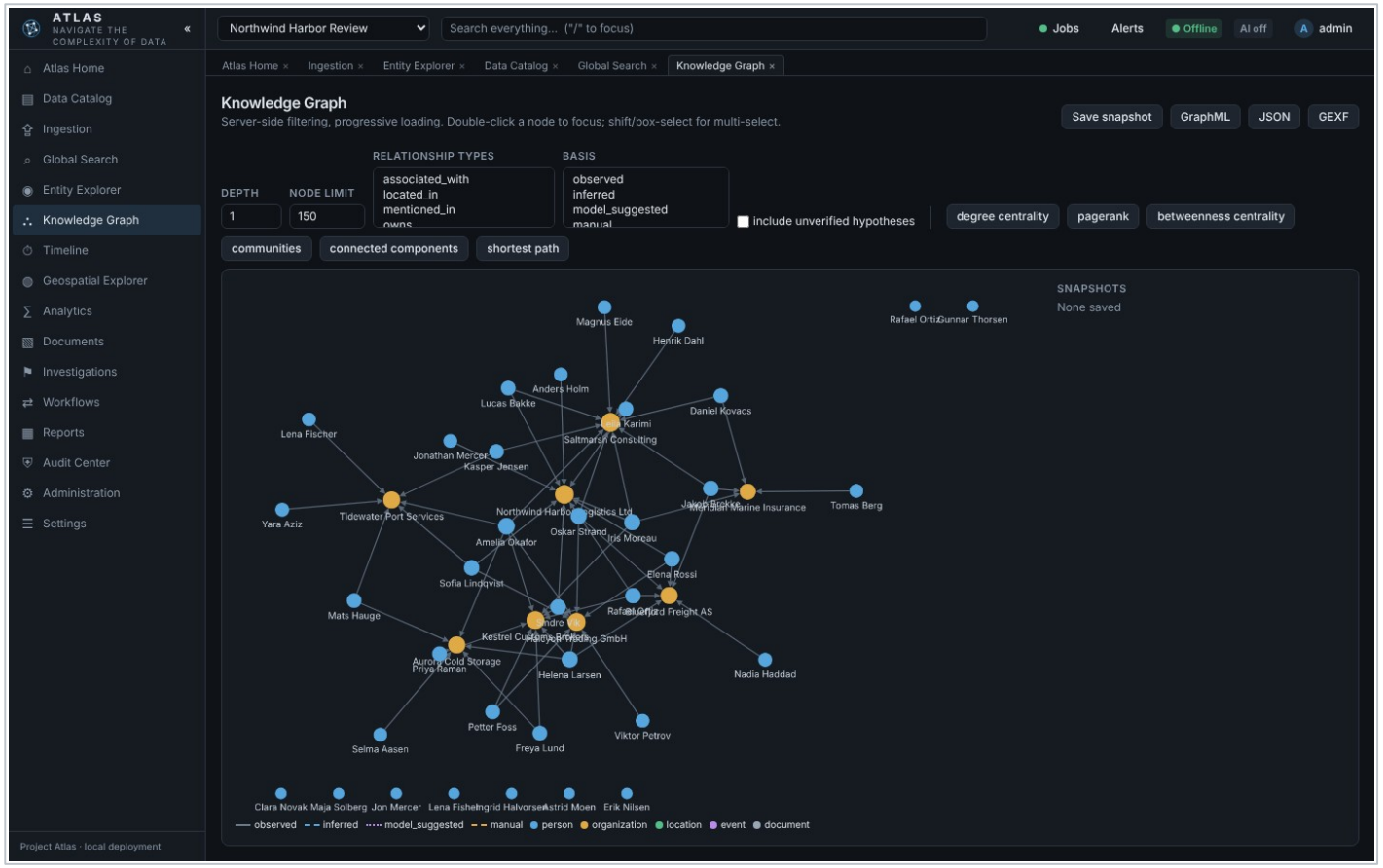
Review
a person accepts or rejects

Missing data is never treated as agreement. Scores are similarity, not probability. Merges are fully revertible.

DEMO PAIR	OUTCOME	WHY
Dr. Amelia Okafor ↔ Amelia Okafor	strong	same e-mail identifier
Jon Mercer ↔ Jonathan Mercer	strong	same phone identifier
Lena Fisher ↔ Lena Fischer	ambiguous	names only — human judgement
Rafael Ortiz ↔ Rafael Ortiz	not strong	phone identifiers conflict
Gunnar Thorsen	no match	nothing shared

Exactly the outcomes in sample-data/expected_results.json — asserted by the test suite.

A knowledge graph honest about its sources



Confidence stays empty unless it was actually measured. A path shows recorded links — not intent.

Every edge carries its basis

- **observed** — stated directly by a source record
- **inferred** — deterministic rule over data
- **model hypothesis** — hidden until a reviewer verifies it
- **manual** — created by a named user

- Degree, PageRank, betweenness, Louvain communities, paths
- Explain connection: per-hop evidence
- Snapshots; GraphML, GEXF, JSON export

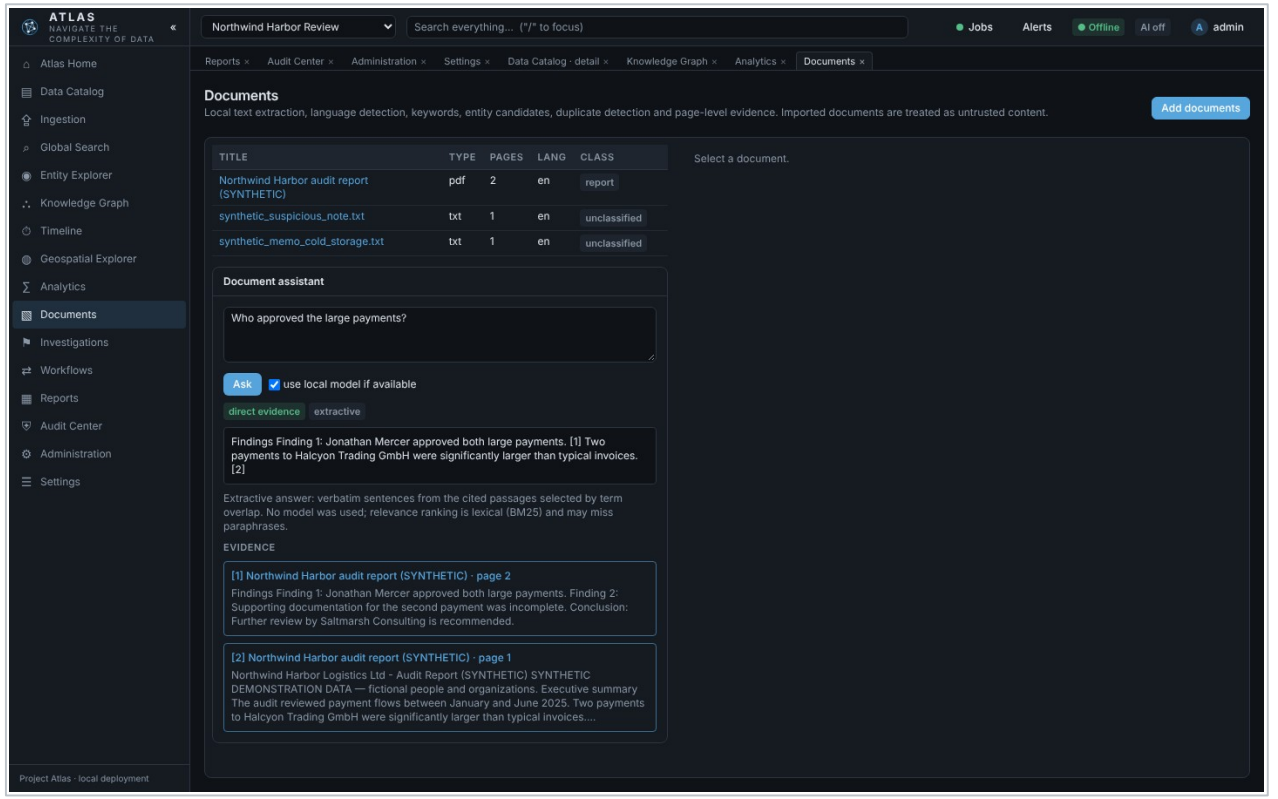
Search, documents and a grounded assistant

Unified search

`"Bluefjord Freight"`	phrase
`Okaf*`	prefix
`Mercer AND payment`	Boolean
`type:entity Okafor`	type filter
`amount>50000`	field filter

Document intelligence

- PDF, DOCX, TXT, Markdown, EML with page-level evidence
- Answers quote and cite pages — or decline
- Embedded instructions flagged, never followed



"Who approved the large payments?" — answered by quoting the synthetic audit report, page 2.

Analytics you can reproduce — and SQL you can trust

Reproducibility by hash

Recorded inputs

version ids + content hashes, parameters, software versions

↓

Run

describe · frequency · trend · correlation · outliers · missing · time series

↓

Result SHA-256

hash of the canonical result, stored with the run

↓

Reproduce

re-run and compare: identical, or changed inputs reported

Demo: outliers T-5007 and T-5023 flagged — with the not-evidence-of-wrongdoing disclaimer — and the reproduced hash is identical.

SQL console: four independent layers

1 Static validation

one SELECT/WITH statement; file, extension and config functions denied

2 Permitted tables only

only readable datasets loaded; sensitive columns dropped

3 Locked engine

in-memory DuckDB; external access off, configuration locked first

4 Resource limits

rows, memory, threads and a wall-clock interrupt

A test proves file access fails even with the validator deliberately bypassed.

Optional local AI, under strict rules

“Ask your data”

You ask

plain-language question

→

Model drafts SQL

validated, never auto-run

→

You review & run

explicit click, sandboxed

→

Atlas computes

deterministic table

→

Model describes

labelled as model output

Local endpoint only

Loopback or private address enforced at startup and on every call. No hosted fallback.

Cited and validated

Answers without valid citations are withheld; SQL is validated; hypotheses are stored unverified.

A person decides

Merges, relationship hypotheses, SQL execution and report text all require explicit action.

Investigations, workflows and reports



Investigations

- Objective, scope and authorization basis
- Verbatim evidence with revisions
- Findings: draft → review → confirmed / disputed / rejected
- No self-review; confirmation needs evidence



Workflows

- 16 typed node types, validated ports
- Preview on 200 rows, no side effects
- Side effects need confirmation
- Failure rolls back data writes



Reports

- PDF, HTML, XLSX, CSV, JSON
- Manifest: versions, parameters, redactions
- SHA-256 of every export
- Formula-injection protection

In the acceptance run, the downloaded PDF bytes matched the manifest SHA-256 exactly.

SECTION 03

Trust and operations

Security, audit, offline assurance, deployment, testing and status

Security enforced on every request

Argon2id passwords
≥ 12 chars, rehash on parameter change

No default account
one-time setup token, deleted after use

Sessions
256-bit tokens, only hashes stored, sliding expiry

Brute force
rate limits + 15-minute lockout after 5 failures

Optional TOTP MFA
admin-issued one-time recovery tokens

404 for non-members
other projects' existence never revealed

PERMISSION	READ-ONLY	REVIEWER	ANALYST	DATA ENG.	PROJ. ADMIN
Read and search	✓	✓	✓	✓	✓
Review matches	·	✓	·	✓	✓
Analyses and SQL	·	✓	✓	✓	✓
Reports, workflows, AI	·	·	✓	✓	✓
Import, resolve, restricted data	·	·	·	✓	✓
Sensitive fields, members	·	·	·	·	✓

System administrators do not automatically see project data. Sensitive fields are redacted in records, SQL, analytics and exports.

Audit, backup and offline assurance



Hash-chained audit

- SHA-256 over previous hash + canonical event
- Triggers block UPDATE and DELETE
- HMAC-signed checkpoints off-database
- Verify reports the first broken sequence



Verified backups

- Online SQLite snapshot + workspace files
- Optional AES-256-GCM (scrypt key)
- Verify: hashes, integrity, row counts
- Restore moves current data aside first



Offline by design

- Socket and DNS guard inside the process
- No telemetry; no external assets
- offline-check with a live probe
- Workflow repeated in a no-network namespace

A test drops the trigger and edits a row: verification reports the exact broken sequence number.

Three ways to deploy the same code

Standalone desktop

PyInstaller app; random loopback port; opens a chromeless browser window; SQLite in the user data folder

Air-gapped workstation

Offline bundle with every wheel and SHA256SUMS; install.sh verifies and installs with --no-index

Private LAN server

Behind a TLS reverse proxy; LAN opt-in and TLS required in production; PostgreSQL in preview

```
pip install -e backend  &&  (cd frontend && npm ci && npm run build)  &&  atlas serve

atlas offline-check      atlas backup --encrypt      atlas verify-backup <file>
```

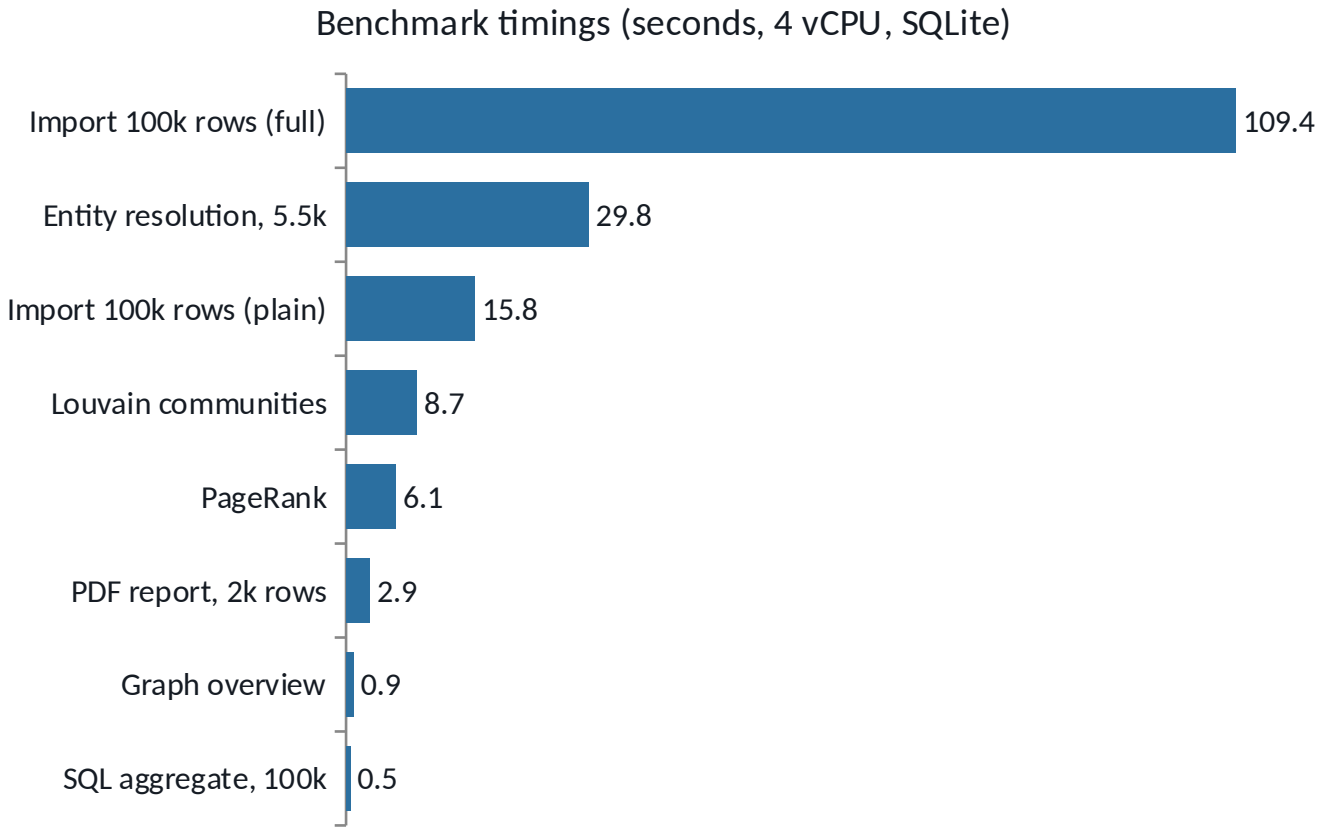
Tested, measured and verified offline

111 backend tests passing

11 frontend unit tests

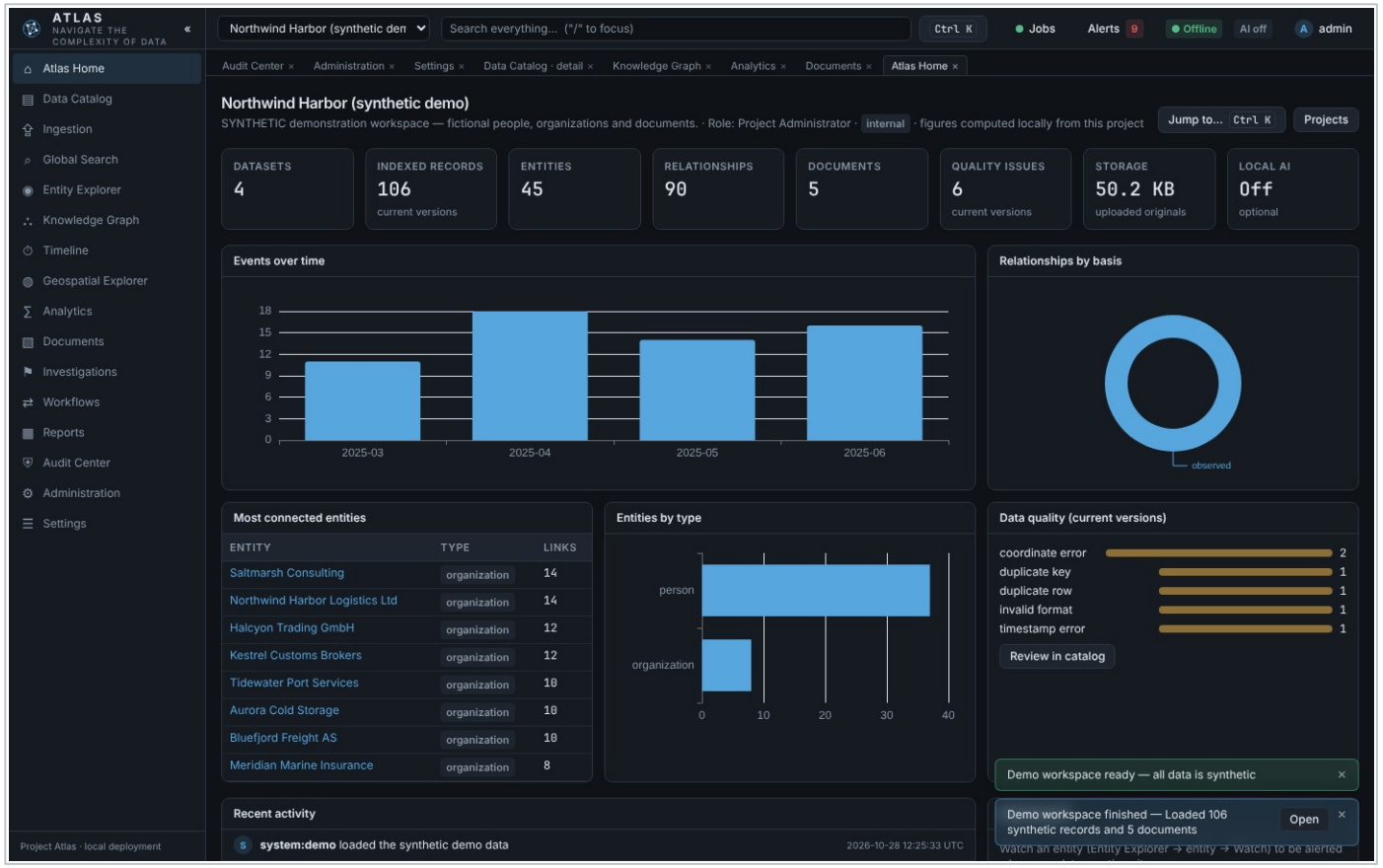
14/14 acceptance steps, incl. no-network phase

E2E Playwright run across all modules



205k records · 5.5k entities · 100k relationships · search p95 374 ms · peak 1.29 GB

Guided demo: Northwind Harbor



All demo data is synthetic (seed 20251027, example.invalid e-mails).

1. Projects → Create demo workspace
2. Ctrl/⌘ K to jump anywhere
3. See the planted quality defects
4. Merge Dr. Amelia Okafor (strong match)
5. Explain a connection in the graph
6. Flag T-5007 and T-5023 as outliers
7. Ask the assistant; export a PDF report

Honest status of release 0.2.0

Complete and tested

- All 16 modules and their APIs
- Ingestion, resolution, graph, search
- Analytics, SQL sandbox, reports
- Audit, backup/restore, offline check
- Linux offline bundle and desktop app

Partial

- AI tested against a mock runtime
- OCR path untested here
- Language detection: 7 languages
- No per-field or per-model grants
- Workflow parameters edited as JSON

Deferred or untested

- PostgreSQL (implemented, untested)
- Windows and macOS packages
- Native Tauri/Electron shell
- Multi-process job workers
- Independent penetration test

Biggest adoption gates for team use: PostgreSQL validation and an independent penetration test.

Suggested roadmap

Next release

- Validate PostgreSQL end to end, add it to CI
- Run the AI assistants against a real local model
- Test OCR; build Windows and macOS packages

Following

- Independent penetration test
- Per-project model allow-lists
- Finer field permissions; workflow parameter forms

Later

- Multi-process job workers
- Resumable mid-file imports
- Polygon operations, richer offline basemaps

Recommendations derived from the engineering report's partial, deferred and risk items — not recorded commitments.

ATLAS

NAVIGATE THE COMPLEXITY OF DATA

Offline. Explainable. Auditable.



Try it: atlas serve → Projects → Create demo workspace

Full detail: docs/overview/Project-Atlas-Technical-Overview.pdf · docs/ in the repository